

Feedforward and Backpropagation

Inclass Project 2 - MA4144

This project contains 12 tasks/questions to be completed, some require written answers. Questions that required written answers are given in blue fonts. Some written questions are open ended, they do not have a correct or wrong answer. You are free to give your opinions, but please provide related answers within the context.

After finishing project run the entire notebook once and **save the notebook as a pdf** (File menu - > Save and Export Notebook As -> PDF). You are **required to upload this PDF on moodle and also the ipynb notebook file as well.**

Name : A.A.W.L.R.Amarasinghe

Index Number : 210031H

Outline of the project

The aim of the project is to build a Multi Layer perceptron (MLP) model from scratch for binary classification. That is given an input x output the associated class label 0 or 1.

In particular, we will classify images of handwritten digits (0, 1, 2, ..., 9). For example, given a set of handwritten digit images that only contain two digits (Eg: 1 and 5) the model will classify the images based on the written digit.

For this we will use the MNIST dataset (collection of 28×28 images of handwritten digits) - you can find additional information about MNIST [here](#).

Use the below cell to use any include any imports

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
```

Section 1: Preparing the data

```
#Load the dataset as training and testing, then print out the shapes  
of the data matrices.  
#The training data will be provided to you.
```

```
data = np.load('train_mnist.npz')
train_X = data['x']
train_y = data['y']
print(train_X.shape)
print(train_y.shape)

(60000, 28, 28)
(60000,)
```

Q1. In the following cell write code to display 5 random images in train_X and it's corresponding label in train_y. Each time it is run, you should get a different set of images. The imshow function in the matplotlib library could be useful. Display them as [grayscale images](#).

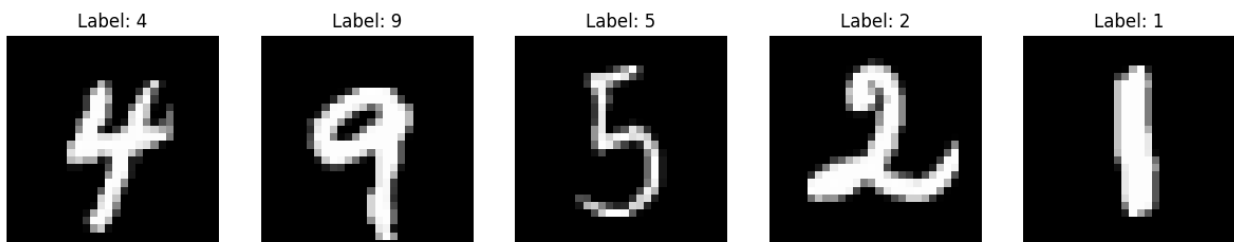
```
#TODO Code to display 5 random handwritten images from train_X and
corresponding labels from train_y

# Randomly select 5 indices
indices = np.random.choice(train_X.shape[0], 5, replace=False)

# Set up the plot
fig, axes = plt.subplots(1, 5, figsize=(15, 3))

for i, idx in enumerate(indices):
    ax = axes[i]
    ax.imshow(train_X[idx].reshape(28, 28), cmap='gray')
    ax.set_title(f"Label: {train_y[idx]}")
    ax.axis('off')

plt.show()
```



Q2. Given two digits d_1 and d_2 , both between 0 and 9, in the following cell fill in the function body to extract all the samples corresponding to d_1 or d_2 only, from the dataset X and labels y . You can use the labels y to filter the dataset. Assume that the label for the i th image $X[i]$ in X is given by $y[i]$. The function should return the extracted samples $X_{\text{extracted}}$ and corresponding labels $y_{\text{extracted}}$. Avoid using for loops as much as possible, infact you do not need any for loops. numpy.where function should be useful.

```
def extract_digits(X, y, d1, d2):

    assert d1 in range(0, 10), "d1 should be a number between 0 and 9"
```

```

inclusive"
    assert d2 in range(0, 10), "d2 should be a number between 0 and 9
inclusive"

    #TODO
    indices = np.where((y == d1) | (y == d2))
    X_extracted = X[indices]
    y_extracted = y[indices]

    return (X_extracted, y_extracted)

```

Q3. Both the training dataset is a 3 dimensional numpy array, each image occupies 28 × 28 dimensions. For convenience of processing data we usually convert each 28 × 28 image matrix to a vector with 784 entries. We call this process **vectorize images**.

Once we vectorize the images, the vectorized data set would be structured as follows: i th row will correspond to a single image and j th column will correspond to the j th pixel value of each vectorized image. However going along with the convention we discussed in the lecture, the input to the MLP model will require that the columns correspond to individual images. Hence we also require a transpose of the vectorized results.

The pixel values in the images will range from 0 to 255. Normalize the pixel values between 0 and 1, by dividing each pixel value of each image by the maximum pixel value of that image. Simply divide each column of the resulting matrix above by the max of each column.

Given a dataset X of size $N \times 28 \times 28$, in the following cell fill in the function to do the following in order;

1. Vectorize the dataset resulting in dataset of size $N \times 784$.
2. Transpose the vectorized result.
3. Normalize the pixel values of each image.
4. Finally return the vectorized, transposed and normalized dataset $X_{transformed}$.

Again, avoid for loops, functions such as `numpy.reshape`, `numpy.max` etc should be useful.

```

def vectorize_images(X):

    #TODO
    # 1. Vectorize the dataset
    X_vectorized = X.reshape(X.shape[0], -1)

    # 2. Transpose the vectorized result
    X_vectorized = X_vectorized.T

    # 3. Normalize the pixel values of each image
    max_values = np.max(X_vectorized, axis=0)
    # Avoid division by zero for images that are all black
    max_values[max_values == 0] = 1
    X_vectorized = X_vectorized / max_values

```

```
return(X_vectorized)
```

Q4. In the following cell write code to;

1. Extract images of the digits $d_1=1$ and $d_2=5$ with their corresponding labels for the training set (train_X, train_y).
2. Then vectorize the data, tranpose the result and normlize the images.
3. Store the results after the final transformations in numpy arrays train_X_1_5, train_y_1_5.
4. Our MLP will output only class labels 0 and 1 (not 1 and 5), so create numpy arrays to store the class labels as follows: $d_1=1 \rightarrow$ class label = 0 and $d_2=5 \rightarrow$ class label = 1. Store them in an array named train_y_1_5.

Use the above functions you implemented above to complete this task. In addition, numpy.where could be useful. Avoid for loops as much as possible.

```
# Extract images of digits 1 and 5
X_1_5, y_1_5 = extract_digits(train_X, train_y, 1, 5)

# Vectorize, transpose and normalize the images
train_X_1_5 = vectorize_images(X_1_5)

# Create binary labels: 1 -> 0, 5 -> 1
train_y_1_5 = np.where(y_1_5 == 1, 0, 1).reshape(1, -1)
```

Section 2: Implementing MLP from scratch with training algorithms.

Now we will implement code to build a customizable MLP model. The hidden layers will have the **Relu activation function** and the final output layer will have **Sigmoid activation function**.

Q5. Recall the following about the activation functions:

1. Sigmoid activation: $y = \sigma(z) = \frac{1}{1 + e^{-z}}$.
2. Derivative of Sigmoid: $y' = \sigma'(z) = \sigma(z)(1 - \sigma(z)) = y(1 - y)$
3. ReLu activation: $y = \text{ReLU}(z) = \max(0, z)$
4. Derivative of ReLu: $y' = \text{ReLU}'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{otherwise} \end{cases} = \begin{cases} 0 & \text{if } y = 0 \\ 1 & \text{otherwise} \end{cases}$

In the following cell implement the functions to compute activation functions Sigmoid and ReLu given z and derivatives of the Sigmoid and ReLu activation functions given y. Note that, in the implementation, the derivative functions should actually accept y as the input not z.

In practice the input will not be just single numbers, but matrices. So functions or derivatives should be applied elementwise on matrices. Again avoid for loops, use the power of numpy arrays - search for numpy's capability of doing elementwise computations.

Important: When implementing the sigmoid function make sure you handle overflows due to e^{-z} being too large. To avoid you can choose to set the sigmoid value to 'the certain appropriate value' if z is less than a certain good enough negative threshold. If you do not handle overflows, the entire result will be useless since the MLP will just output Nan (not a number) for every input at the end.

```
def sigmoid(Z):
    # Sigmoid activation:  $y = 1 / (1 + e^{-z})$ 
    Z = np.clip(Z, -500, 500)
    sigma = 1 / (1 + np.exp(-Z))
    return sigma

def deriv_sigmoid(Y):
    # Derivative of Sigmoid:  $y' = y * (1 - y)$ 
    sigma_prime = Y * (1 - Y)
    return sigma_prime

def ReLu(Z):
    # ReLU activation:  $y = \max(0, z)$ 
    relu = np.maximum(0, Z)
    return relu

def deriv_ReLu(Y):
    relu_prime = (Y > 0).astype(float)
    return relu_prime
```

Q6. The following piece of code defines a simple MLP architecture as a Python class and subsequent initialization of a MLP model. Certain lines of code contains commented line numbers. Write a short sentence for each such line explaining its purpose. Feel free to refer to the lecture notes or any resources to answers these question. In addition, explain what the Y, Z, W variables refer to and their purpose

```
class NNet:
    def __init__(self, input_size = 784, output_size = 1, batch_size =
1000, hidden_layers = [500, 250, 50]):
        self.Y = []
        self.Z = []
        self.W = []
        self.input_size = input_size
        self.output_size = output_size
        self.batch_size = batch_size
        self.hidden_layers = hidden_layers

        layers = [input_size] + hidden_layers + [output_size]
        L = len(hidden_layers) + 1

        for i in range(1, L + 1):
            self.Y.append(np.zeros((layers[i], batch_size)))
#line1
```

```

        self.Z.append(np.zeros((layers[i], batch_size)))
#Line2
        self.W.append(np.random.normal(0, np.sqrt(2.0/layers[i-1]), (layers[i], layers[i-1] + 1))) #Line3

```

Questions (write answers in the cell below as strings)

(i) What does the Y, Z, W variables refer to and their purpose?

(ii) Line1: Explanation

(iii) Line2: Explanation

(iv) Line3: Explanation

#Write answers here as strings:

```

ans_i = "Y refers to the activations (outputs) of the neurons at each layer," \
" Z refers to the weighted input to each layer (before activation), " \
" W refers to the weights between the layers that are learned during training."

```

```

ans_ii = "Line 1 initializes an array to store the activations of the current layer. " \
" The shape is (layers[i], batch_size), where layers[i] is the number of neurons in the layer and batch_size is the number of examples processed together."

```

```

ans_iii = "Line 2 initializes an array to store the weighted inputs to the current layer." \
" The shape is (layers[i], batch_size), and it stores the sum of the weighted inputs before applying the activation function."

```

```

ans_iv = "Line 3 initializes the weights for the current layer with random values between -0.5 and 0.5. " \
"The shape of the weight matrix is (layers[i], layers[i-1] + 1) to account for the number of neurons in the previous layer and the bias term."

```

Q7. Now we will implement the feedforward algorithm. Recall from the lectures that for each layer l there is input $Y^{(l-1)}$ from the previous layer if $l > 1$ and input data X if $l = 1$. Then we compute $Z^{(l)}$ using the weight matrix $W^{(l)}$ as follows from matrix multiplication:

$$Z^{(l)} = W^{(l)} Y^{(l-1)}$$

Make sure that during multiplication you add an additional row of one's to $Y^{(l-1)}$ to accommodate the bias term (concatenate the row of ones as the last row to be consistent with the grader). However, the rows of ones should not permanently remain on $Y^{(l-1)}$. Explain what

the bias term is and how adding a row of one's help with the bias terms. The weight matrices are initialised to afford this extra bias term, so no change to either $Z^{(l)}$ or $W^{(l)}$ is needed.

Next compute $Y^{(l)}$, the output of layer l by activation through sigmoid.

$$Y^{(l)} = \sigma(Z^{(l)})$$

The implemented feedforward algorithm should take in a NNet model and an input matrix X and output the modified MLP model - the Y 's and Z 's computed should be stored in the model for the backpropagation algorithm.

As usual, avoid for loops as much as possible, use the power of numpy. However, you may use a for loop to iterate through the layers of the model.

```
def feedforward(model, X):  
  
    #TODO  
    Y_prev = X # Input to the first layer is the data itself  
  
    for l in range(len(model.hidden_layers) + 1):  
        # Add bias term to the input of the current layer  
        Y_prev_with_bias = np.concatenate((Y_prev, np.ones((1,  
Y_prev.shape[1]))), axis=0)  
  
        # Compute Z for the current layer  
        model.Z[l] = model.W[l] @ Y_prev_with_bias  
  
        # Compute Y for the current layer (output of the layer)  
        if l < len(model.hidden_layers): # Hidden layers use ReLU  
            model.Y[l] = ReLU(model.Z[l])  
        else: # Output layer uses Sigmoid  
            model.Y[l] = sigmoid(model.Z[l])  
  
        Y_prev = model.Y[l] # Set the output of the current layer as  
input for the next  
  
    return(model)
```

Question (write answer in the cell below as a string)

Explain what the bias term is and how adding a row of one's help with the bias terms.

#Write the answer here as a string:

```
ans_bias = "The bias term is an additional parameter in a neural  
network that allows the model to make predictions even when the input  
is zero. " \  
"It shifts the activation function and enables the network to fit the  
data better. " \  
"Adding a row of ones to the input matrix allows the bias to be
```

incorporated into the matrix multiplication by treating the bias as part of the weight matrix." \n "This ensures that it is included in the computation without needing separate handling."

Q8. Now we will implement the backpropagation algorithm. The cost function C at the end is given by the square loss.

$C = \frac{1}{2} \|Y^{(L)} - Y\|^2$, where $Y^{(L)}$ is the final output vector of the feedforward algorithm and Y is the actual label vector associated with the input X .

At each layer $l = 1, 2, \dots, L$ we compute the following (note that the gradients are matrices with the same dimensions as the variable to which we derivating with respect to):

1. Gradient of C with respect to $Z^{(l)}$ as $\frac{\partial C}{\partial Z^{(l)}} = \text{deriv}(A^{(l)}(Z^{(l)})) \odot \frac{\partial C}{\partial Y^{(L)}}$, where $A^{(l)}$ is the activation function of the l th layer, and we use the derivative of that here. The $\odot$ refers to the elementwise multiplication.
2. Gradient of C with respect to $W^{(l)}$ as $\frac{\partial C}{\partial W^{(l)}} = \frac{\partial C}{\partial Z^{(l)}} (Y^{(l-1)})^T$ this is entirely matrix multiplication.
3. Gradient of C with respect to $Y^{(l-1)}$ as $\frac{\partial C}{\partial Y^{(l-1)}} = (W^{(l)})^T \frac{\partial C}{\partial Z^{(l)}}$ this is also entirely matrix multiplication.
4. Update weights by: $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial C}{\partial W^{(l)}}$, where $\eta > 0$ is the learning rate.

The loss derivative (the gradient of C with respect to $Y^{(L)}$) at the last layer is given by:

$$\frac{\partial C}{\partial Y^{(L)}} = Y^{(L)} - Y$$

By convention we consider $Y^{(0)} = X$, the input data.

Based on the backpropagation algorithm implement the backpropagation method in the following cell. Remember to temporarily add a row of ones to $Y^{(l-1)}$ (as the last row to be consistent with the grader) when computing $\frac{\partial C}{\partial W^{(l)}}$ as we discussed back in the feedforward algorithm. Make sure you avoid for loops as much as possible.

The function takes in a NNet model, input data X and the corresponding class labels Y . learning rate can be set as desired.


```

def backpropagation(model, X, Y, eta = 0.01):
    #TODO
    L = len(model.hidden_layers) + 1 # Total number of layers

    # Loss derivative at the output layer
    dC_dYL = model.Y[L-1] - Y

    # Backpropagate through the layers in reverse order
    for l in range(L, 0, -1):
        # Compute dC/dZ(l)
        if l == L: # Output layer uses Sigmoid
            dC_dZl = deriv_sigmoid(model.Y[l-1]) * dC_dYL # Element-
            wise multiplication
        else: # Hidden layers use ReLU
            dC_dZl = deriv_ReLu(model.Y[l-1]) * dC_dYL # Element-wise
            multiplication

        # Compute dC/dW(l)
        if l == 1: # First layer input is X
            Y_prev_with_bias = np.concatenate((X, np.ones((1,
X.shape[1]))), axis=0)
        else: # Previous layers' output with bias
            Y_prev_with_bias = np.concatenate((model.Y[l-2],
np.ones((1, model.Y[l-2].shape[1]))), axis=0)

        dC_dWl = dC_dZl @ Y_prev_with_bias.T # Matrix multiplication

        # Compute dC/dY(l-1) for backpropagating to the previous layer
        # Exclude the bias term from the weight matrix transpose
        dC_dYL = model.W[l-1][:, :-1].T @ dC_dZl

        # Update weights
        model.W[l-1] -= eta * dC_dWl

    return(model)

```

Q9. Now implement the training algorithm.

The training method takes in training data X , actual label Y , number of epochs, batch_size, learning rate $\eta > 0$. The training will happen in epochs. For each epoch, permute the data columns of both X and Y , then divide both X and Y into mini batches each with the given batch size. Then run the feedforward and backpropagation for each such batch iteratively.

At the end of each iteration, keep track of the cost C and the l_2 -norm of change in each weight matrix $W^{(l)}$.

At the end of the last epoch, plot the variation cost C and change in weight matrices. Then return the trained model.

```

def train_NNet(X, Y, epochs = 20, batch_size = 1000, eta = 0.01,
hidden_layers = [128, 64]):

    # Initialize model with custom architecture
    model = NNet(input_size=X.shape[0], output_size=Y.shape[0],
batch_size=batch_size, hidden_layers=hidden_layers)

    costs = []
    weight_changes = []
    n_samples = X.shape[1]

    for epoch in range(epochs):
        # Permute data
        perm = np.random.permutation(n_samples)
        X_shuffled = X[:, perm]
        Y_shuffled = Y[:, perm]

        epoch_cost = 0
        epoch_weight_change = 0

        # Mini-batch training
        for i in range(0, n_samples, batch_size):
            end_idx = min(i + batch_size, n_samples)
            X_batch = X_shuffled[:, i:end_idx]
            Y_batch = Y_shuffled[:, i:end_idx]

            # Store previous weights for change calculation
            prev_weights = [W.copy() for W in model.W]

            # Feedforward and backpropagation
            model = feedforward(model, X_batch)
            model = backpropagation(model, X_batch, Y_batch, eta)

            # Calculate cost
            batch_cost = 0.5 * np.mean((model.Y[-1] - Y_batch)**2)
            epoch_cost += batch_cost

            # Calculate weight changes
            batch_weight_change = sum(np.linalg.norm(model.W[l] -
prev_weights[l])
                                for l in range(len(model.W)))
            epoch_weight_change += batch_weight_change

        costs.append(epoch_cost)
        weight_changes.append(epoch_weight_change)

    # Plot results
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(costs)

```

```

ax1.set_title('Cost over Epochs')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Cost')

ax2.plot(weight_changes)
ax2.set_title('Weight Changes over Epochs')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Weight Change (L2 norm)')

plt.tight_layout()
plt.show()

return(model)

```

Section 3: Evaluation using test data

Q10. Implement the following function predict. Given test images (3 dimensional numpy array that contains 28×28 digit images) it will correctly recognize the written digits between d1 and d2. You can assume that test_images will only contain images of digits d1 and d2. Inside predict you would need to preprocess the images using vectorize, predict the correct labels using model and then output the correct labels. Output should be a vector predicted of d1's and d2's, predicted[i] should correspond to the test_image[i].

```

def predict(model, test_images, d1, d2):

    # Vectorize test images
    X_test = vectorize_images(test_images)

    # Run feedforward
    model = feedforward(model, X_test)

    # Get predictions (sigmoid output > 0.5 means class 1, else class 0)
    binary_predictions = (model.Y[-1] > 0.5).astype(int).flatten()

    # Convert binary predictions back to original digit labels
    predicted = np.where(binary_predictions == 0, d1, d2)

    return(predicted)

```

Q11. Use the train_NNet function to train a MLP model to classify between images of digits 1 and 5. An accuracy ≈ 99 is achievable. Test with different batch sizes, η values and hidden layers. Find which of those hyperparameters gives the best test accuracy. Name this model, model_1_5. This model will be tested on unseen test data within the grader. So make sure you train the best possible model. The grader will use your own predict function to evaluate the model.

```

# Train model for digits 1 and 5
print("Training model for digits 1 and 5...")

```

```

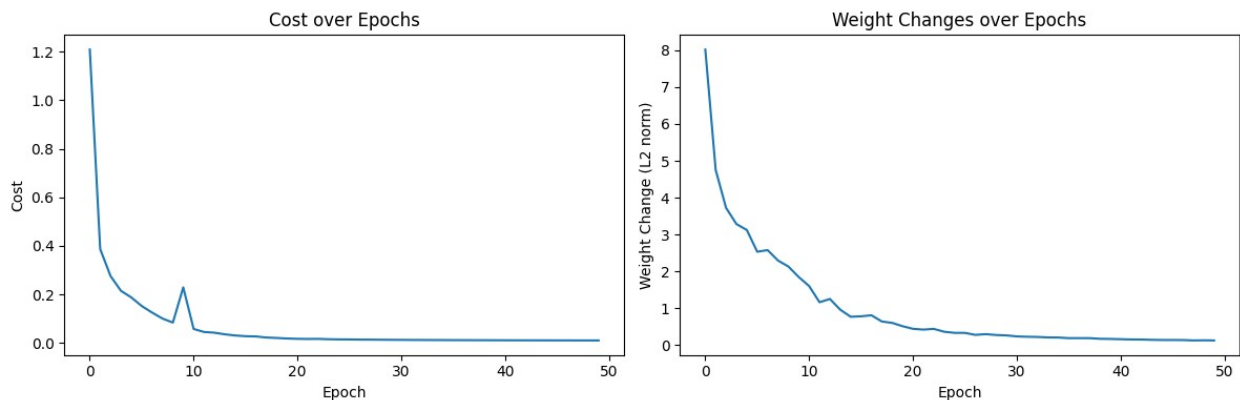
model_1_5 = train_NNet(train_X_1_5, train_y_1_5, epochs=50,
batch_size=64, eta=0.01)

# Test accuracy on training data
train_predictions = predict(model_1_5, X_1_5, 1, 5)
train_accuracy = accuracy_score(y_1_5, train_predictions)
print(f"Training accuracy for digits 1 and 5: {train_accuracy:.4f}")

model_1_5 = model_1_5

Training model for digits 1 and 5...

```



Training accuracy for digits 1 and 5: 0.9999

Q12. Do the same as in Q11 with the digits 7 and 9 Name this model, model_7_9. This model will be tested on unseen test data within the grader. So make sure you train the best possible model. The grader will use your own predict function to evaluate the model.

```

# Extract and prepare data for digits 7 and 9
X_7_9, y_7_9 = extract_digits(train_X, train_y, 7, 9)
train_X_7_9 = vectorize_images(X_7_9)
train_y_7_9 = np.where(y_7_9 == 7, 0, 1).reshape(1, -1)

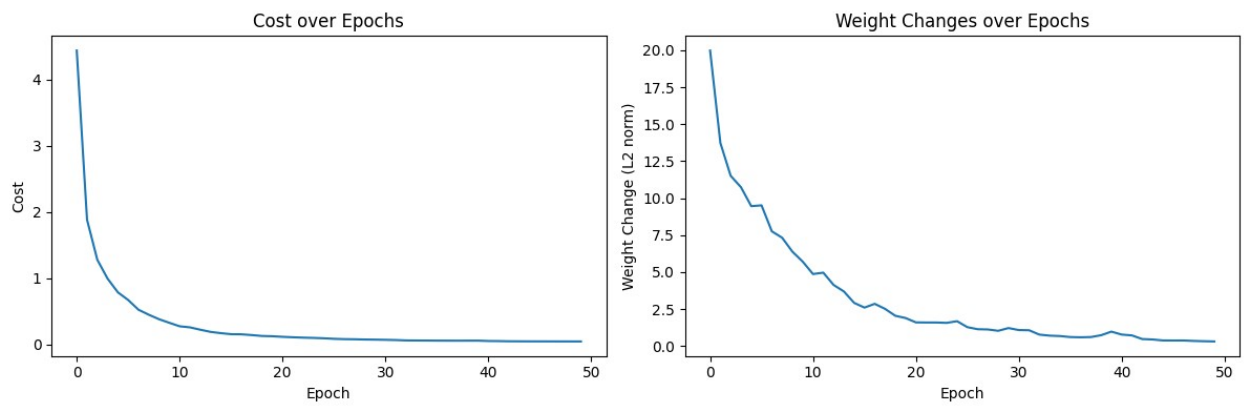
# Train model for digits 7 and 9
print("Training model for digits 7 and 9...")
model_7_9 = train_NNet(train_X_7_9, train_y_7_9, epochs=50,
batch_size=64, eta=0.01)

# Test accuracy on training data
train_predictions_7_9 = predict(model_7_9, X_7_9, 7, 9)
train_accuracy_7_9 = accuracy_score(y_7_9, train_predictions_7_9)
print(f"Training accuracy for digits 7 and 9:
{train_accuracy_7_9:.4f}")

model_7_9 = model_7_9

```

Training model for digits 7 and 9...



Training accuracy for digits 7 and 9: 0.9996